

Build 4: Agent Testing Log

Jacob Poore and Arian Zarazua

April 24, 2026

Overview

We tested the Build 4 HITL Router Agent using a game-stats CSV (5,980 rows, NFL seasons 2013–2023, 35 columns) covering per-game offensive statistics for all 32 NFL teams. We ran 7 prompts total: 4 passed and 3 failed.

Prompt Log

#	Route	Result	Notes
1	Tool	Pass	Correct tool and args
2	Tool	Fail	No team filter; aggregated all teams
3	Codegen	Fail	Wrong team names; chart is empty
4	Tool	Pass	Summarized market column correctly
5	Codegen	Fail	Summed a string column; axis labels malformed
6	Tool	Pass	Heatmap saved correctly
7	Tool	Fail	<code>plot_missingness()</code> argument conflict error

Table 1: Summary of prompts. Prompts: (1) plot avg passing yards by season; (2–3) plot NFC East rushing and passing yards by season; (4) summarize a random categorical variable; (5) plot market value by team for 2023; (6) correlation heatmap for offensive metrics; (7) missingness patterns across performance metrics.

Test Details

Test 1 – Passing yards over seasons [Pass]

Prompt: Plot average passing yards by season.

The router selected `plot_temporal_line_chart` with the correct arguments. The chart shows mean passing yards per game ranging from about 235 to 259 yards across 2013–2023, with a dip in 2017 and a decline after 2021.

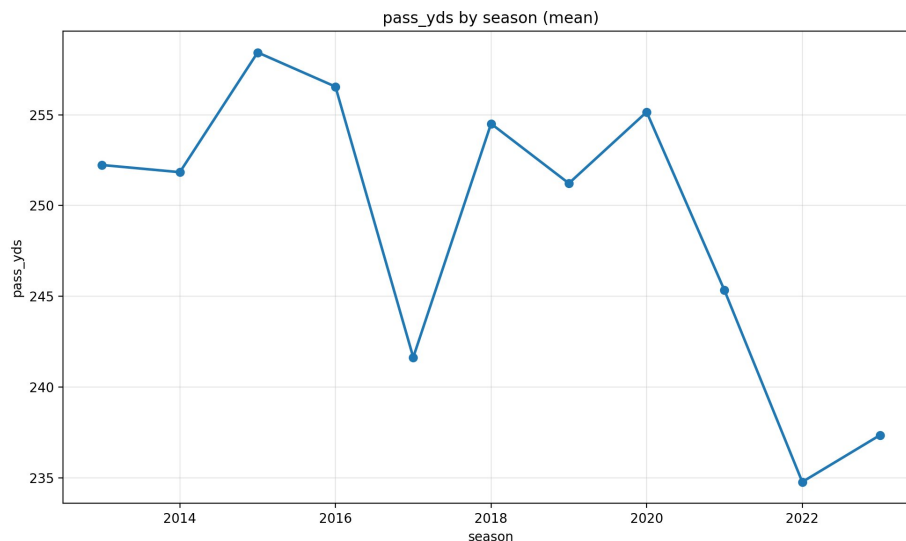


Figure 1: Mean passing yards per game by season (2013–2023).

Test 2 – NFC East yards by season via tool route [Fail]

Prompt: Plot total rushing and passing yards for every team in the NFC East by season.

The router sent this to `aggregate_by_temporal_column` with `aggregation_method = sum`. The tool returned 11 season groups but aggregated the entire dataset with no team filter. The output shows league-wide totals rather than per-team NFC East figures.

What went wrong: The tool does not support filtering by team or division. This request needed codegen so a filter on the `alias` column could be applied first.

Test 3 – NFC East yards by season via codegen [Fail]

Prompt: (follow-up via `code` command) Plot total rushing and passing yards for every team in the NFC East by season.

The agent generated a script filtering on `alias` using full team names (e.g., "Dallas Cowboys"). The script exited with code 0, but the chart is blank. The `alias` column uses abbreviated identifiers, so no rows matched the filter.

What went wrong: The agent did not check what values `alias` actually contains before writing the filter. Running `summarize_categorical` on `alias` first would have revealed the correct identifiers.

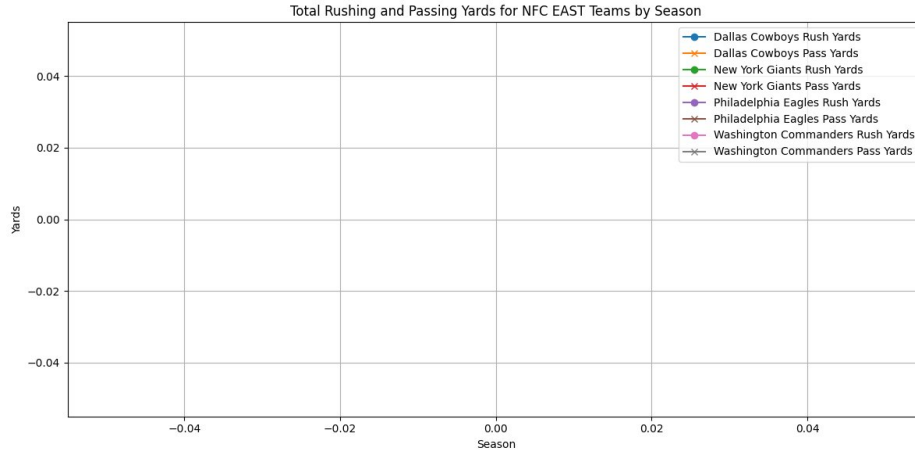


Figure 2: NFC East rushing and passing yards by season. The chart is empty because the team name filter matched zero rows.

Test 4 – Summarize categorical variable [Pass]

Prompt: Summarize a random categorical variable.

The router selected `summarize_categorical` and chose the `market` column. Results: 5,980 entries, 0 missing, 30 unique markets. Top values were Los Angeles (374) and New York (361).

Test 5 – Market value by team for 2023 [Fail]

Prompt: Plot market value by team for 2023 data.

The agent generated code that filtered to `season == 2023`, grouped by `alias`, and called `.sum()` on the `market` column. The script exited cleanly, but `market` is a string column containing city names. Summing it concatenated the strings, producing axis labels like `ArizonaArizona` and `DallasDallasDallas`.

What went wrong: No numeric market-value column exists in the dataset. The agent should have flagged the missing column and asked for clarification instead of running code on the wrong column type.



Figure 3: Attempted market value by team for 2023. The `market` column holds city name strings; summing it produced concatenated labels and an unreadable chart.

Test 6 – Correlation heatmap [Pass]

Prompt: Show a correlation heatmap for offensive metrics.

The router selected `plot_corr_heatmap` and the tool ran correctly across all 27 numeric columns. Strong positive correlations appear among passing volume metrics (`pass_att`, `pass_cmp`, `pass_yds`) and between `penalties` and `penalty_yds`. `pass_int` shows a moderate negative correlation with `passer_rating`.

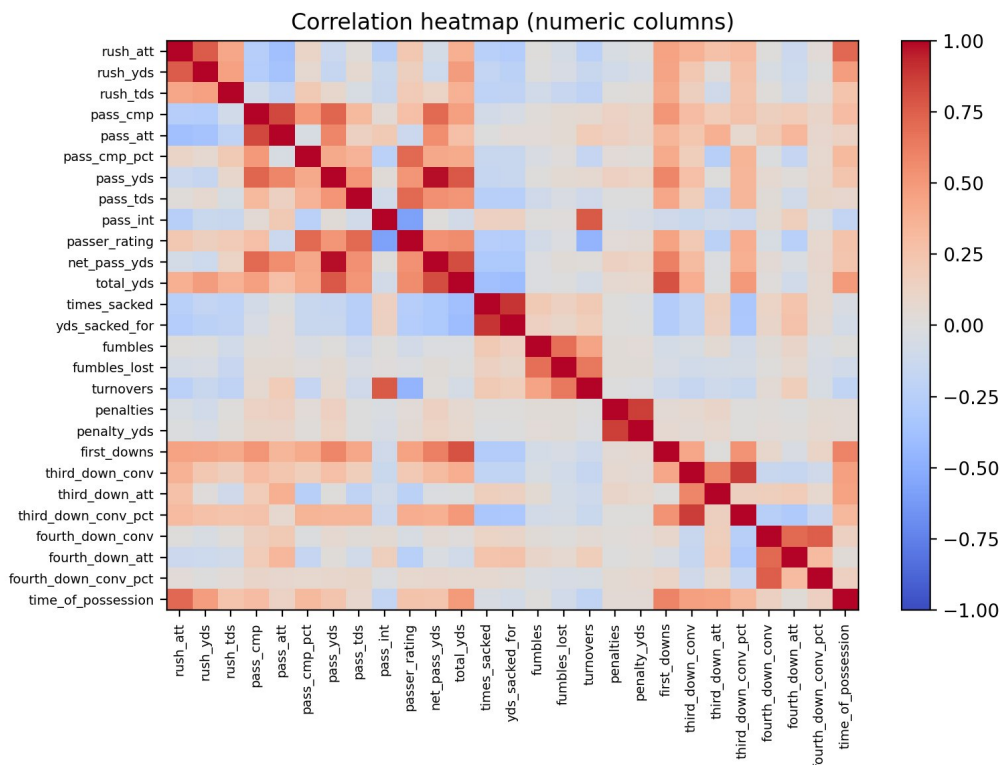


Figure 4: Correlation heatmap across all 27 numeric columns.

Test 7 – Missingness patterns [Fail]

Prompt: What patterns of missingness exist in the performance metrics across different seasons?

The router selected `plot_missingness` and the user approved. The tool threw:

ERROR running tool: `plot_missingness()` got multiple values for argument 'miss_df'

What went wrong: The router passed `miss_df` as both a positional and keyword argument, causing a `TypeError`. The router also passed an empty dict rather than a computed missingness DataFrame. This is the same argument-construction bug seen in Build 3 (Tests 4 and 9).

Langfuse Tracing Verification

Langfuse tracing was active throughout the session. The export contains 31 trace spans covering every agent step: `build-router`, `build-run-tool`, `build-summarize`, `build-codegen`, and `build-suggest`. The Langfuse router traces confirm the RAG output was used.

RAG chunk retrieval – representative traces

The following are four representative RAG retrievals from the session, drawn directly from terminal output.

Trace: plot average passing yards by season

Langfuse span id: 95289029c24185c40b809e4180758d81 (18:36:46 UTC)

```
[Chunk 1 | source=nfl_analysis_guides\basic_nfl_data_analysis_workflow.md
          | section=Step 2: map to dataset | score=0.5769]
Stats, Snapcounts, Scoring, ExpectedPoints, Seasons, Roster.
```

```
[Chunk 2 | source=datasets_notes\Seasons\season_analysis_workflow.md
          | section=Step 1: aggregate games | score=0.5057]
team-season grouping.
```

```
[Chunk 3 | source=nfl_analysis_guides\basic_nfl_data_analysis_workflow.md
          | section=Step 5: compute summaries | score=0.5151]
production, efficiency, turnovers, usage.
```

Router output consumed from RAG context:

```
{"mode":"tool","tool":"plot_temporal_line_chart",
  "args":{"temporal_column":"season","numeric_column":"pass_yds"}}
```

Trace: NFC East rushing and passing yards by season

Langfuse span id: d043a6ad50e18cbe9a8e1caa0745a4d2 (18:37:56 UTC)

```
[Chunk 1 | source=nfl_analysis_guides\basic_nfl_data_analysis_workflow.md
          | section=Step 2: map to dataset | score=0.5333]
Stats, Snapcounts, Scoring, ExpectedPoints, Seasons, Roster.
```

```
[Chunk 2 | source=datasets_notes\Seasons\season_analysis_workflow.md
          | section=Step 1: aggregate games | score=0.5173]
team-season grouping.
```

```
[Chunk 3 | source=nfl_analysis_guides\team_strength_analysis.md
          | section=Step 1: aggregate season or games | score=0.4854]
group by team.
```

Router output consumed from RAG context:

```
{"mode":"tool","tool":"aggregate_by_temporal_column",
  "args":{"temporal_column":"season",
          "numeric_columns":["rush_yds","pass_yds"],
          "aggregation_method":"sum"}}
```

Trace: summarize a random categorical variable

Langfuse span id: 7b6409f89e6b43ffedde3e1ad0caff34 (18:42:26 UTC)

```
[Chunk 1 | source=build0_tool_notes\summarize_categorical.md  
          | section=Purpose | score=0.5464]
```

Summarize categorical variables using counts and common values.

```
[Chunk 2 | source=build0_tool_notes\summarize_categorical.md  
          | section=Inputs | score=0.5365]
```

- df - cat_cols - optional top_k for most frequent categories

```
[Chunk 3 | source=analysis_guides\tool_selection_rules.md  
          | section=Dataset overview requests | score=0.5331]
```

If the user asks to describe or summarize the dataset,
start with a basic profile and variable-type split.

Router output consumed from RAG context:

```
{"mode":"tool","tool":"summarize_categorical",  
  "args":{"column":"market"}}
```

Trace: missingness patterns across performance metrics

Langfuse span id: cbedcc2e1e34a07ccb43fe34831d4aa4 (18:41:12 UTC)

```
[Chunk 1 | source=nfl_analysis_guides\basic_nfl_data_analysis_workflow.md  
          | section=Step 2: map to dataset | score=0.5302]
```

Stats, Snapcounts, Scoring, ExpectedPoints, Seasons, Roster.

```
[Chunk 2 | source=datasets_notes\Roster\depth_chart_reasoning.md  
          | section=Step 4: usage trends | score=0.5112]
```

changes over season.

```
[Chunk 3 | source=datasets_notes\Seasons\season_analysis_workflow.md  
          | section=Step 4: consistency | score=0.5099]
```

variance across games.

Router output consumed from RAG context:

```
{"mode":"tool","tool":"plot_missingness",  
  "args":{"miss_df":{},"out_path":""}}
```

Observations

RAG retrieval fired on every router call and returned 3–4 chunks per query. Chunk scores ranged from roughly 0.48 to 0.58 across all queries. The top-scoring source across most queries was `basic_nfl_data_analysis_workflow.md`, which appears to be a broadly relevant document.

Overall, RAG had no meaningful positive impact on agent performance in this testing session. Although Langfuse traces confirm that retrieval occurred on every router call and that the router consumed RAG context, the retrieved chunks were often too general to correct the agent’s actual failure modes. The agent still made errors involving missing filters, incorrect assumptions about column values, wrong variable types, and malformed tool arguments. For example, RAG helped retrieve broad workflow guidance such as grouping by team-season or summarizing categorical variables, but it did not prevent the agent from aggregating all teams instead of filtering to the NFC East, using full team names when the dataset used abbreviations, summing the string-valued `market` column, or passing invalid arguments to `plot_missingness`. Therefore, we assess RAG as having no clear impact on performance rather than enhancing it. To improve RAG performance, the knowledge base should include more concrete dataset-specific documentation, including exact column names, valid categorical values, division/team mappings, tool argument schemas, and examples of correct router outputs for common prompt types. The router should also be instructed to use retrieved context as a validation step before selecting a tool or generating code, especially for checking whether requested variables exist, whether columns are numeric or categorical, and whether a tool supports the required filtering operation.